



Fase 2 do Projeto de Computação Gráfica

3º ano Licenciatura Engenharia Informática

Computação Gráfica

Universidade do Minho

2025/2026

Grupo 11

Diogo Alves Ferreira (A106904)

Hugo Araújo Cunha (A106808)

José Miguel da Silva Santos (A72443)

Mariana Vivas Rodrigues (A106898)

1. Introdução

Este trabalho foi realizado no âmbito da segunda fase do projeto da unidade curricular Computação Gráfica.

Na primeira fase foi proposta a criação da *engine* e do *generator*. Nesta etapa, o motor gráfico desenvolvido anteriormente foi estendido de forma a interpretar ficheiros XML com grupos aninhados, permitindo associar a cada grupo um conjunto de transformações geométricas, modelos e subgrupos. Desta forma, passou a ser possível representar uma cena como uma árvore, em que cada grupo herda as transformações aplicadas pelos seus antecessores. Esta abordagem é particularmente adequada para modelar estruturas compostas por várias partes relacionadas entre si, como é o caso do sistema solar, cena pedida no enunciado para demonstração desta fase.

2. Descrição do Problema

A segunda fase deste projeto tem como objetivo o desenvolvimento de novas funcionalidades aplicadas ao motor gráfico desenvolvido na fase anterior. Este deve agora processar transformações geométricas, nomeadamente translações, rotações e escalas, descritas no ficheiro de configuração em formato XML.

Ao contrário da fase anterior, em que a cena era composta essencialmente por modelos carregados a partir de ficheiros .3d, nesta etapa a organização da cena passa a ser hierárquica. Cada grupo pode conter transformações, modelos e outros grupos, sendo que os grupos filhos herdam automaticamente as transformações aplicadas nos grupos pais.

O ficheiro XML deve, nesta fase, representar um modelo estático do sistema solar, incluindo o Sol, os planetas e satélites. Assim, o motor deve ser capaz de:

- Ler os grupos definidos no XML;
- Armazenar em memória as transformações e os modelos associados a cada grupo;
- Carregar os vértices dos ficheiros .3d;
- Desenhar os modelos aplicando corretamente as transformações e respeitando a hierarquia da cena.

Para além disso, foi necessário adaptar também o gerador, introduzindo uma nova primitiva geométrica:

ring radius outradius slices file.3d

Esta primitiva corresponde a uma coroa circular centrada na origem, situada no plano xz e desenhada de forma a ser visível dos dois lados.

Relativamente ao modelo do sistema solar, os requisitos principais consistem em:

- utilizar primitivas adequadas para representar os vários corpos celestes;
- organizar a cena de forma hierárquica;
- aplicar transformações de translação, rotação e escala para posicionar e dimensionar os diferentes elementos.

3. Gerador

De forma a demonstrar devidamente o modelo do sistema solar, necessitou-se de atualizar o Gerador desenvolvido na fase anterior. Posto isto, decidiu-se criar uma primitiva que calcula os pontos necessários para desenhar uma coroa circular, que vamos usar para representar saturno de uma maneira mais fidedigna.

3.1. Ring

O anel, composto por triângulos desenhados no sentido positivo do eixo y e no sentido negativo do eixo y , é centrado na origem sobre os eixos x e z . Este anel tem um raio interior (*radius*), um raio exterior (*outradius*) e um número de slices pelo qual vai ser dividido.

$$\alpha = \frac{2\pi}{slices} \times a, a \in [0, slices[$$

Na implementação realizada, os dois raios são recebidos diretamente no construtor da classe Ring, isto é:

- ri corresponde ao raio interior;
- re corresponde ao raio exterior.

Recorrendo à distância do ponto à origem (ri no caso dos pontos do raio interior e re no caso dos pontos do raio exterior) e ao ângulo α , podem calcular-se as coordenadas x e z dos pontos que formam o anel, sendo que a coordenada y tem sempre valor 0.

As fórmulas utilizadas são:

$$\alpha = \frac{2\pi}{slices} \times a, a \in [0, slices[$$

$$x = dist \times \sin(\alpha)$$

$$z = dist \times \cos(\alpha)$$

Para cada divisão angular são considerados dois ângulos consecutivos:

$$\alpha_1 = \frac{2\pi}{slices} \times a$$

$$\alpha_2 = \frac{2\pi}{slices} \times (a + 1)$$

Com base nesses valores, são calculados quatro pontos:

- p_{1i} — ponto do raio interior no ângulo α_1
- p_{2i} — ponto do raio interior no ângulo α_2
- p_{1e} — ponto do raio exterior no ângulo α_1
- p_{2e} — ponto do raio exterior no ângulo α_2

Ou seja, as coordenadas são obtidas por:

- $x_{1i} = ri \times \sin(\alpha_1)$
- $z_{1i} = ri \times \cos(\alpha_1)$
- $x_{2i} = ri \times \sin(\alpha_2)$
- $z_{2i} = ri \times \cos(\alpha_2)$
- $x_{1e} = re \times \sin(\alpha_1)$
- $z_{1e} = re \times \cos(\alpha_1)$
- $x_{2e} = re \times \sin(\alpha_2)$
- $z_{2e} = re \times \cos(\alpha_2)$

Depois de calculados estes quatro pontos, são gerados os triângulos que formam o anel.

Para a face visível de $y > 0$, são desenhados os triângulos (p_{1i}, p_{2e}, p_{1e}) e (p_{1i}, p_{2i}, p_{2e}) .

Para a face visível de $y < 0$, são desenhados os triângulos (p_{1i}, p_{1e}, p_{2e}) e (p_{1i}, p_{2e}, p_{2i}) .

Desta forma, cada slice origina quatro triângulos, permitindo que o anel seja visível de ambos os lados

4. Engine

4.1. Estrutura hierárquica da cena

Para representar a cena em memória foi utilizada uma estrutura hierárquica baseada na classe *Group*. Desta forma, a cena é organizada sob a forma de uma árvore, em que cada nó representa um grupo e os respectivos subgrupos herdam as transformações aplicadas pelos seus antecessores.

Neste caso, cada grupo guarda:

- um vetor `transforms`, onde são guardados ponteiros para objetos derivados da classe abstrata `Transformation`;
- um vetor `models`, onde são guardados os modelos do tipo `Figure` associados ao grupo;
- um vetor `children`, onde ficam armazenados os ponteiros para os subgrupos.

4.2. Leitura recursiva do XML

A leitura do ficheiro XML é realizada no arranque da aplicação, tal como pedido no enunciado. Após a leitura dos parâmetros da janela e da câmara, a engine identifica o grupo raiz da cena e inicia o processamento recursivo dos elementos `<group>`.

Sempre que é encontrado um novo grupo, é criado um objeto `Group`, sendo depois analisados os seus elementos filhos. Quando o elemento corresponde a `<transform>`, são criados os respectivos objetos de translação, rotação ou escala; quando corresponde a `<models>`, são carregados os ficheiros `.3d` associados; quando corresponde a um novo `<group>`, o processo repete-se recursivamente.

Posto isto, para ler o ficheiro XML foi implementado o método `loadXML(...)`, pertencente à classe `Config`. Este método carrega o documento XML, obtém o elemento `<world>` e faz a leitura dos parâmetros globais da cena, nomeadamente:

- as dimensões da janela;
- a posição da câmara;
- o ponto `lookAt`;
- o vetor `up`;
- os parâmetros de projeção (`fov`, `near` e `far`).

Depois disso, o método procura o primeiro elemento `<group>` existente dentro de `<world>`. Caso esse grupo exista, é criado o grupo raiz da cena através do ponteiro `root`, sendo depois chamado o método privado `parseGroup(...)`, responsável por processar recursivamente toda a hierarquia.

O método `parseGroup(...)` percorre todos os filhos de um grupo e trata cada caso consoante o tipo de elemento encontrado:

- se encontrar um bloco `<transform>`, percorre as transformações aí definidas e cria objetos das classes *Translate*, *Rotate* ou *Scale*, que são guardadas no vetor `transforms` do grupo atual;
- se encontrar um bloco `<models>`, percorre os vários elementos `<model>`, lê o atributo `file`, carrega o respetivo ficheiro `.3d` através do método `load(...)` da classe *Figure* e guarda o modelo no vetor `models`;

- se encontrar outro elemento <group>, cria um novo grupo filho, adiciona-o ao vetor children e chama novamente parseGroup(...), permitindo assim continuar o parsing recursivo.

4.3. Aplicação das transformações geométricas

Para desenhar a cena, a árvore de grupos é percorrida recursivamente. O método draw() de cada grupo começa por guardar o estado atual da matriz de transformação com glPushMatrix(). A seguir, são aplicadas as transformações do grupo pela ordem em que foram lidas do XML. Depois disso, são desenhados os modelos associados ao grupo e, por fim, são visitados os grupos filhos. No final do processo, é invocado glPopMatrix(), restaurando o referencial anterior.

Desta forma, a organização hierárquica definida no ficheiro XML é respeitada durante o desenho da cena. A utilização de glPushMatrix() e glPopMatrix() permite isolar o contexto geométrico de cada grupo, assegurando assim que cada conjunto de modelos e subgrupos é desenhado no sistema de coordenadas que lhe corresponde.

Como as transformações são armazenadas no vetor transforms segundo a ordem em que aparecem no XML, a sua aplicação durante o desenho respeita exatamente a sequência definida no ficheiro de configuração.

4.4. Herança de transformações

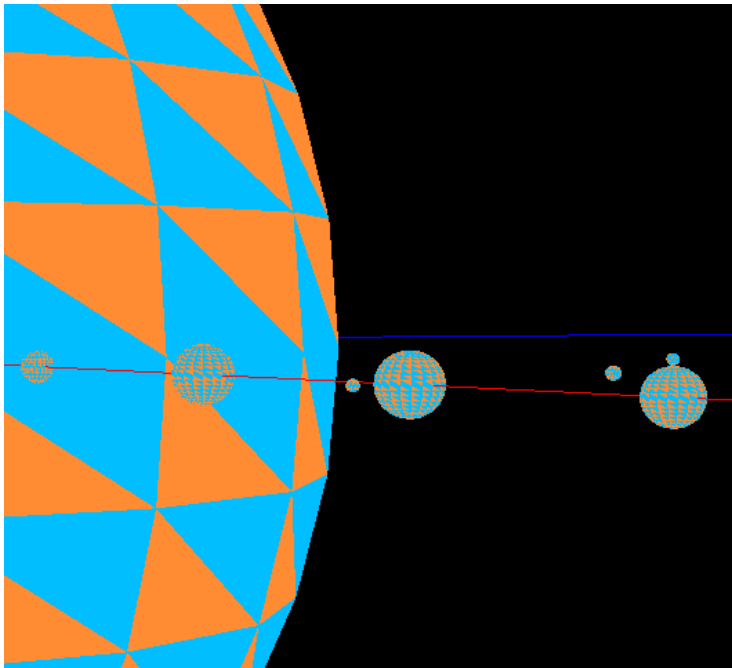
A herança de transformações entre grupos é garantida pela pilha de matrizes do OpenGL. Como os grupos filhos são desenhados depois de aplicadas as transformações do grupo pai, passam automaticamente a ser desenhados no novo referencial. Isto permite representar estruturas hierárquicas de forma natural, como por exemplo um planeta relativamente ao Sol, ou uma lua relativamente ao planeta a que pertence.

4.5. Visualização e modos de desenho

Para ajudar na visualização das primitivas geométricas geradas, foi temporariamente introduzido um mecanismo de coloração cíclica dos triângulos no método *Figure::draw()*. Em vez de desenhar todos os modelos com uma única cor, os triângulos passam a ser desenhados alternadamente com duas cores distintas. Percorremos o array de vértices de 3 em 3 pontos (a cada triângulo) e aplicando $\frac{i}{3} \bmod 2$, permite-nos ir alternando as cores aplicadas a cada triângulo.

Foi também corrigido o mecanismo de alterar entre os diferentes modos de desenho. Na versão inicial, as teclas associadas aos modos preenchido, linha e ponto não produziam o efeito esperado, uma vez que a variável mode, utilizada em *renderScene()*, mantinha sempre o valor inicial. Após a correção, essa variável

passou a ser atualizada diretamente no método `keyboard(...)`, ficando assim assegurado o funcionamento correto da mudança entre `GL_FILL`, `GL_LINE` e `GL_POINT`.



5. Demo scene

5.1. Organização da cena

Para a *demo scene* da Fase 2 foi construída uma representação estática do sistema solar, utilizando a estrutura hierárquica implementada na engine. O Sol foi colocado no centro da cena, sendo os planetas definidos como subgrupos com transformações próprias de translação, rotação e escala. Em alguns casos, foram ainda definidos subgrupos adicionais para representar luas ou outros elementos associados (caso de luas, ou dos anéis de Saturno).

5.2. Uso da hierarquia

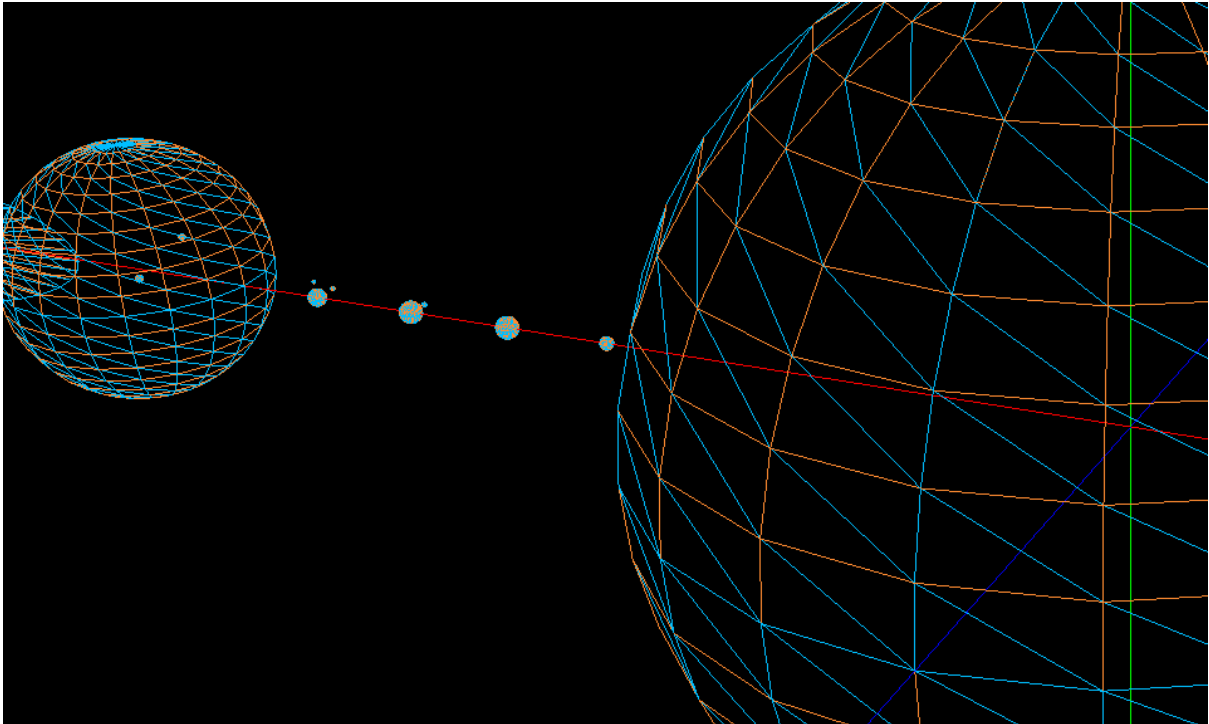
A utilização de grupos hierárquicos revelou-se particularmente adequada para esta cena, uma vez que permite modelar naturalmente relações de dependência espacial. Por exemplo, uma lua pode ser definida como subgrupo de um planeta, o que faz com que qualquer transformação aplicada ao planeta influencie automaticamente a posição relativa da lua.

5.3. Transformações geométricas aplicadas aos elementos do sistema solar

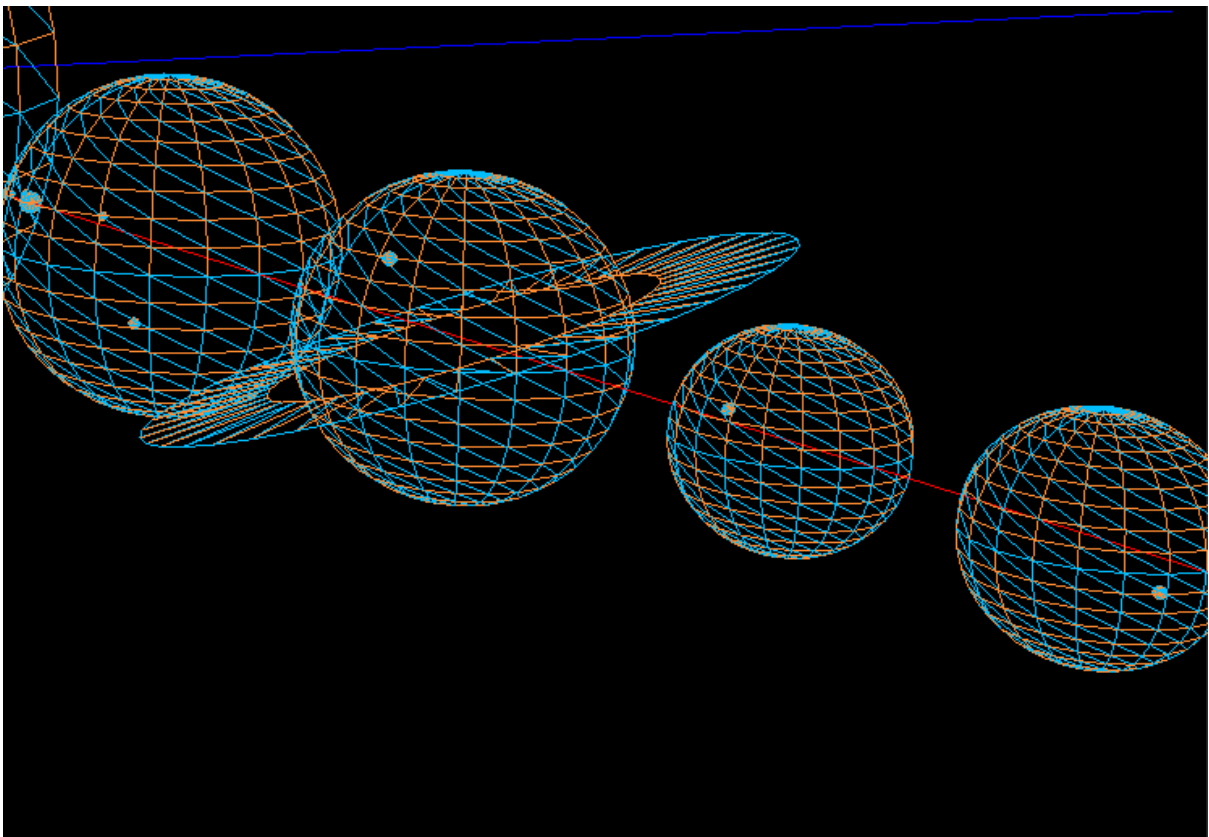
De maneira a resumir a organização do modelo do sistema solar, é possível ver na tabela seguinte o conjunto de transformações geométricas associado a cada elemento da cena.

Nome	Escala(x,y,z)	Translate	Rotação
Sol	(20,20,20)	-	-
Mercúrio	(0.3, 0.3, 0.3)	(24, 0, 0)	180
Venus	(0.5, 0.5, 0.5)	(29, 0, 0)	180
Terra	(0.5, 0.5, 0.5)	(34, 0, 0)	180
Lua	(0.1, 0.1, 0.1)	(0, 0, 1)	180
Marte	(0.4, 0.4, 0.4)	(39, 0, 0)	180
Fobos	(0.1,0.1,0.1)	(-0.5, 0.3, 0.5)	-
Deimos	(0.08, 0.08, 0.08)	(0.5, 0.5, 0.5)	-
Jupiter	(6, 6, 6)	(49, 0, 0)	180
Europa	(0.15, 0.15, 0.15)	(-1, 1, 2)	-
Ganimedes	(0.18, 0.18, 0.18)	(3, -2, 3)	-
Saturno	(5, 5, 5)	(66, 0, 0)	180
Tita	(0.2, 0.2, 0.2)	(2, 3, 4)	-
Anel de Saturno	(1, 1, 1)	-	20
Urano	(3, 3, 3)	(80, 0, 0)	180
Titania	(0.15, 0.15, 0.15)	(0, 1, 2)	-
Neptuno	(3, 3, 3)	(90, 0, 0)	180
Tritão	(0.15, 0.15, 0.15)	(0, -1.5, -2)	-

5.4. Imagem da cena



Sistema solar entre o sol a júpiter onde é possível observar algumas das luas



Sistema solar entre júpiter e neptuno onde é possível observar o restante sistema solar e as suas luas assim como os anéis de saturno

6. Conclusão

Nesta segunda fase do projeto foi possível estender o motor gráfico desenvolvido anteriormente, acrescentando-lhe suporte para transformações geométricas hierárquicas e para a leitura de cenas organizadas em grupos a partir de um ficheiro XML.

A principal evolução relativamente à fase anterior foi a introdução de uma estrutura em árvore, baseada na classe Group. Conseguimos assim associar a cada grupo um conjunto de transformações, modelos e subgrupos. Esta abordagem tornou possível representar cenas mais complexas e organizadas, em que os grupos filhos herdam automaticamente as transformações aplicadas nos grupos pais.

Para além disso, foi adaptado o processo de parsing do XML, passando o motor a interpretar não apenas os modelos da cena, mas também a sua estrutura hierárquica e as transformações geométricas associadas. O desenho recursivo da árvore de grupos, em conjunto com a utilização de `glPushMatrix()` e `glPopMatrix()`, permitiu garantir a correta aplicação e herança das transformações ao longo da cena.

Por fim, a construção do modelo estático do sistema solar permitiu validar o funcionamento global da solução implementada. A utilização de esferas para os corpos celestes e de um anel para Saturno, combinada com transformações de escala, translação e rotação, demonstrou que o motor é agora capaz de representar cenas hierárquicas de forma correta e estruturada.

Num futuro gostaríamos de implementar uma câmara fps assim como uma nova forma geométrica(esfera low-poly) para podermos desenhar a cintura de asteroides de uma maneira mais fiável.